

为了让你们两个人开发时**绝对不撞车、不抢同一个文件、不互相等待**，最好的分工策略是**“垂直组件切割 (Vertical Component Slicing)”**。

简单来说：你在代码库里建两个独立文件夹，你只写 **app/core** 和 **app/api**，Molly 只写 **app/ai**。你们之间通过唯一的**“Python 函数签名 (接口契约)”**通信。

以下是为您和 Molly 量身定制的 **V0.5 MVP 模块化极速分工方案**：

🧱 模块一：网关、存储与飞书接入（由你负责 —— Dev A）

定位：修路的人（系统骨架、数据库、飞书交互、HTTP 接口）**专属代码目录：** **app/core/**, **app/db/**, **app/api/**, **app/integrations/**

你的具体开发任务：

- Web 接口与路由 (app/api/v1/)**
 - 编写全部 RESTful 路由（登录、资料导入、资产审核、画像增删改查、会话、方案生成）。
 - 负责出参入参的 HTTP 拦截、全局错误码（**FEISHU_PERMISSION_DENIED** 等）拦截器。
 - 第一天产出 openapi.json 扔给前端。**
- 飞书适配器 (app/integrations/feishu.py)**
 - 实现飞书 Mock 登录（返回 Cookie），后续留好真实 OAuth 替换槽位。
 - 调通飞书 Open API：根据链接拉取飞书文档纯文本内容与作者元数据。
- 数据库与持久化 (app/db/)**
 - 使用 SQLAlchemy 建好 6 张主表（Users, Sources, Profiles, Assets, Sessions, SolutionRuns）。
 - 实现结构化数据的基本 CRUD 接口（给前端审核打回、修改用）。
- 异步任务与轮询 (app/core/jobs.py)**
 - 配置 Redis + FastAPI BackgroundTasks。
 - 编写 **/jobs/{job_id}** 轮询状态更新逻辑，负责在数据库更新 **status: fetching -> completed**。

🧠 模块二：AI 引擎、RAG 与算法护栏（由 Molly 负责 —— Dev B）

定位：造车的人（Prompt、Pydantic Schema、向量检索、算子函数）**专属代码目录：** **app/ai/**（包含 **schemas/**, **prompts/**, **retrieval/**, **pipelines/**）

Molly 的具体开发任务：

- Pydantic 结构化数据契约 (app/ai/schemas/)**
 - 定义 **CustomerProfileSchema**（画像结构）。
 - 定义 **ExperienceSchema** 和 **CapabilitySchema**（经验与原子能力卡片）。
 - 定义 **SolutionSchema**（方案 8 大区块 + 来源 ID 绑定）。
- 信息提取算子 (app/ai/pipelines/extractor.py)**
 - 编写 Prompt，写函数 **extract_profile_from_text(text: str)**。
 - 编写 Prompt，写函数 **extract_assets_from_text(text: str, type: str)**。
 - 配合 **Instructor** 库做硬校验，确保提取出来的全是合规 JSON。
- 混合检索算子 (app/ai/retrieval/search.py)**

- 配置本地 ChromaDB 或 pgvector。
 - 编写向量化入库函数 `index_asset(asset_id, text, tags)`。
 - 编写检索函数 `search_hybrid_assets(query, profile_tags, top_k)`。
4. 方案生成与质检算子 (`app/ai/pipelines/generator.py`)
- 编写最核心的方案生成 Prompt。
 - 编写函数 `generate_quick_solution(profile, matched_assets)`。
 - 实现服务端质检：核对 AI 吐出的 `source_id` 是否在检索列表中，不在则强行标记为 `ai_inference` (AI 推断)。

💖 你们俩之间的“通信契约”(Day 1 上午就定死这个)

你们俩不需要去管对方的代码怎么写的，只需要在 Day 1 开工前，在 `app/ai/interface.py` 里写下这 4 个 Python 函数签名：

```
# app/ai/interface.py （你们俩的接口桥梁）

# 1. Molly 提供给你：提取客户画像
async def ai_extract_profile(raw_text: str) -> dict: ...

# 2. Molly 提供给你：提取资产（经验/原子能力）
async def ai_extract_assets(raw_text: str, asset_type: str) -> list[dict]: ...

# 3. 你提供给 Molly：向向量库灌入已被人工审核通过的资产
async def ai_index_verified_asset(asset_id: str, content: str, tags: list[str]): ...

# 4. Molly 提供给你：核心方案生成（最重要的 API）
async def ai_generate_quick_solution(profile_json: dict, session_history: list) -> dict: ...
```

协作机制极其简单：

- **Molly 的工作：** 她的代码只要保证这 4 个函数输入文本、输出正确的字典，她的任务就完成了。她可以用控制台（Terminal）脚本独立测试自己的代码，**完全不需要启动飞书或 Web 服务！**
- **你的工作：** 你的 FastAPI 收到前端请求后，直接 `await ai_generate_quick_solution(...)` 调 Molly 的函数。拿到结果后写进 PostgreSQL 数据库并返回给前端！

📅 3天冲刺交付进度节点（8.5 - 8.7）

时间	你的进度 (Dev A)	Molly 的进度 (Dev B)	两人解耦验证动作
Day 1 (8.5) 契约与 脚手架	1. Docker 拉起 Postgres + Redis。 2. 建表模型定义完成。 3. 导出 <code>openapi.json</code> 发给前端。	1. 准备 10 条手写 JSON 假数据。 2. 调通 Pydantic 模型。 3. 写好 Prompt 基础类。	晚上合并： 通过 Mock 接口，前端能调通登录并拿到假数据。

时间	你的进度 (Dev A)	Molly 的进度 (Dev B)	两人解耦验证动作
Day 2 (8.6) 逻辑与 算子	1. 写好飞书文档抓取 API。 2. 写好 CRUD 审核接口（通 过/打回）。 3. 写好异步任务轮询框架。	1. 跑通 <code>ai_extract_profile</code> 。 2. 跑通 <code>search_hybrid_assets</code> 检 索。 3. 在控制台调通 <code>ai_generate_quick_solution</code> 。	晚上合并： 你调 Molly 的真实函 数，实现“粘贴文本 -> 后台生成画像 JSON 落库”。
Day 3 (8.7) 闭环与 联调	1. 串联完整 API：输入问题 - > 调 Molly 方案生成 -> 保存 方案到 DB。 2. 统一全局错误码和异常拦 截。	1. 加入质检逻辑（校验 <code>source_id</code> ）。 2. 调优 Prompt，确保无匹配时优雅输 出“边界缺口”。	晚上合并： 黄金流程完全跑通！ 演示“输入痛点 -> 极 速吐出带溯源的方 案”。

- 💡 给你们俩的 Vibe Coding 避坑心法：
1. **绝对不要修改对方目录里的代码：** 每个人在自己的目录里是绝对的“国王”。如果要调对方的逻辑，直接调 `interface.py` 里的函数。

2. **用 Git 分支隔离：**
 - 你在 `git checkout -b feat-gateway` 上开发。
 - Molly 在 `git checkout -b feat-ai-engine` 上开发。
 - 每天晚上 8 点，两个人坐在一起，把代码 Merge 到 `main` 分支。

3. **Molly 不需要等待你的数据库：** Molly 在写 RAG 和生成逻辑时，可以直接在本地放一个 `fake_assets.json` 文件来模拟数据库。等 Day 3 再把你真实 PostgreSQL 查出来的资产传给她。

这套分工方式在黑客松和短周期冲刺中被验证过无数次，**清爽、高效、无冲突**，今晚就按照这个模块方案去分配任务吧！**完全不影响！GitHub 本身就是托管 Git 分支的平台，用 GitHub 配合分支隔离，才是最正宗、最安全的使用方式。**

用 GitHub + 分支隔离，不仅不会增加负担，反而有以下巨大好处：

1. 你的电脑炸了代码也在（GitHub 云端备份）。

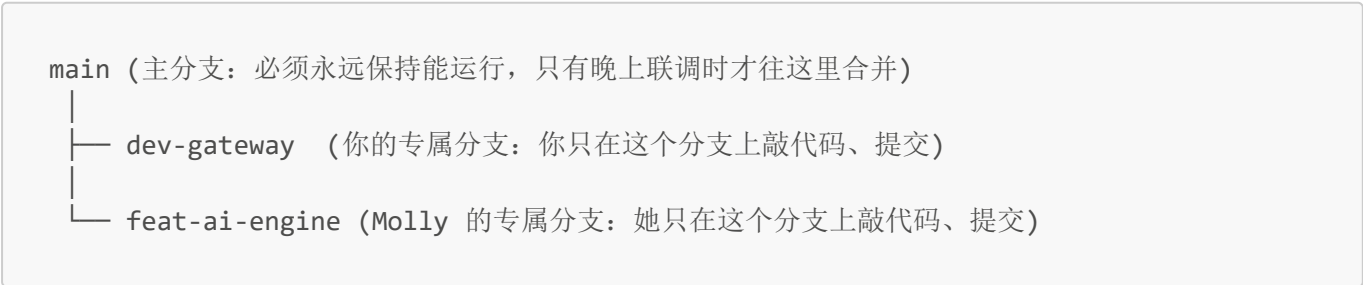
2. 两个人绝对不会把对方写好的代码覆盖掉。

3. 每天晚上可以用 **Pull Request (PR)** 优雅地合并代码，一眼就能看到对方今天写了什么。

为了让你们 13 天开发不被 Git 搞晕，我为你们量身定制了一套 “**GitHub 极简三人/双人流 (3-Branch Workflow)**”，只要遵循几条傻瓜命令，保准你们 0 冲突！

🌲 一、GitHub 仓库分支结构设计（只需要 3 个分支）

仓库里**永远只保留这 3 个分支**：



🔧 二、手把手操作指南（从 Day 1 到 Day 3）

第一步：建仓库与初始分支（Day 1 上午，由你操作一次）

你建好项目脚手架后，推送到 GitHub：

```
# 1. 在你的电脑上初始化项目
git init
git add .
git commit -m "feat: 项目初始化脚手架与目录结构"

# 2. 关联 GitHub 远程仓库并推送到 main 分支
git remote add origin https://github.com/你的用户名/taodaobao-engine.git
git branch -M main
git push -u origin main

# 3. 创建你自己的分支并推送到 GitHub
git checkout -b dev-gateway
git push -u origin dev-gateway
```

同时，让 Molly 从 GitHub 克隆项目，并创建她的分支：

```
# Molly 在她电脑上执行：
git clone https://github.com/你的用户名/taodaobao-engine.git
cd taodaobao-engine

# 创建并切换到她的专属分支
git checkout -b feat-ai-engine
git push -u origin feat-ai-engine
```

第二步：日常开发循环（白天各自敲代码，重复执行）

你在你的电脑上（只在 **dev-gateway** 分支）：

```
# 改完代码后，随时提交并推送到 GitHub 备份
git add .
git commit -m "feat: 完成模拟登录和数据库用户表"
git push origin dev-gateway
```

Molly 在她的电脑上（只在 **feat-ai-engine** 分支）：

```
# 改完代码后，随时提交并推送到 GitHub 备份
git add .
```

```
git commit -m "feat: 完成画像提取 Prompt 和 Pydantic Schema"
git push origin feat-ai-engine
```

💡 **为什么不会冲突？** 因为你把代码推到了云端的 `dev-gateway`，她推到了 `feat-ai-engine`，你们俩在 GitHub 上走的是两条平行线，根本碰不到一起！

第三步：每晚合并代码（Day 1~3 晚上 8 点，合体时间）

每天晚上，你们俩坐在一起，把各自今天写好的代码合并到 `main` 主分支：

方式 A（最直观，推荐）：在 GitHub 网页上合并（Pull Request）

1. 打开 GitHub 你的仓库页面，点击 "Pull requests" -> "New pull request"。
2. 基准分支（base）选择： `main`，比较分支（compare）选择： `dev-gateway`。
3. 点击 "Create pull request"，然后再点 "Merge pull request"。
4. Molly 也在网页上操作一次，把 `feat-ai-engine` 合并到 `main`。

合并完后，两个人必须同步最新的 `main` 代码到本地：

```
# 1. 切换回 main 分支并拉取最新合并的代码
git checkout main
git pull origin main

# 2. 切换回自己的分支，把 main 的最新代码融进自己的分支
git checkout dev-gateway # (Molly 则是 checkout feat-ai-engine)
git merge main
```

🔵 三、保证“0 冲突”的黄金两条铁律

只要遵守这两条，你们这 13 天绝对不会遇到恼人的冲突（Merge Conflict）：

铁律 1：目录物理隔离（最重要！）

- 你绝对不修改 `app/ai/` 目录下的任何文件。
- Molly 绝对不修改 `app/api/`、`app/db/` 目录下的任何文件。
- 原理：Git 非常智能，如果两个人修改的是完全不同文件夹里的文件，哪怕同时合并到 `main`，Git 也会自动 100% 无缝融合，绝对不会报错！

铁律 2：绝不直接在 `main` 分支上敲代码

- `main` 分支是用来展示和演示给组长看的“净土”。
- 平时敲代码之前，先看一下终端命令行，确保自己处于 `dev-gateway` 或 `feat-ai-engine` 分支上。

💡 总结：你和 Molly 今天需要记下的口诀

1. 开工前： `git checkout` 自己的分支

2. 写完后: `git add . -> git commit -m "说明" -> git push origin` 自己的分支
3. 晚上合代码: 去 GitHub 网页上发 PR 合到 `main`, 然后本地 `git checkout main && git pull`。

就这么简单! GitHub 就是你们的“代码保险箱”和“合并中转站”, 放心地用起来吧!